

Artificial Intelligence

Lab 03 - Searching Problems (1)

AI Course Team

Eng. Yousef Elbaroudy - Eng. Sahar Mohammed

Eng. Reham Ibrahim - Eng. Mawada Ashraf

Eng. Eman Nasser

2025/2026

In [1]:
```python
# Make a Graph class
class Graph:
    def __init__(self, root=None, weighted=None, directed=None):
        # Adjacency list representation
        if root:
            self.__graph = {root:[]}
        else:
            self.__graph = {"root":[]}

        self.__weighted = weighted
        if weighted == True:
            self.__weighted = weighted
        elif self.__weighted != None:
            raise Exception("weighted parameter should be True or None")

        self.__directed = False
        if directed == True:
            self.__directed = directed
        elif self.__directed == None:
            raise Exception("directed parameter should be True or None")

        print(f"Graph is created with root name: {list(self.__graph.keys())[0]}")

    #_____________________________________________________________________________________________
    def add_vertex(self, vertex):
        if vertex not in self.__graph.keys():
            self.__graph[vertex] = []
            print("Vertex is added successfully !")
        else:
            print("This vertex does exist in the graph already !")
    #_____________________________________________________________________________________________
    def add_edge(self, vertex_1, vertex_2, weight=None):
        if vertex_1 in self.__graph.keys() and vertex_2 in self.__graph.keys():
            if not self.__directed:
                if not self.__weighted:
                    self.__graph[vertex_1].append(vertex_2)
                    self.__graph[vertex_2].append(vertex_1)
                else:
                    if weight != None:
                        self.__graph[vertex_1].append((vertex_2, weight))
                        self.__graph[vertex_2].append((vertex_1, weight))
                    else:
                        raise Exception("It should be a weight for the edge")
            else:
                if not self.__weighted:
                    self.__graph[vertex_1].append(vertex_2)
                else:
                    if weight != None:
                        self.__graph[vertex_1].append((vertex_2, weight))
                    else:
                        raise Exception("It should be a weight for the edge")
        else:
```

```
            raise Exception("It should be both vertices exist in the graph !")

        print(f"Edge added between {vertex_1} and {vertex_2}")

    #_____________________________________________________________________________________________
    def get_graph(self):
        return self.__graph

    #_____________________________________________________________________________________________
    def DFS(self, goal, start=None):
        if start != None:
            lst = [[start]]
        else:
            lst = [["root"]]

        visited = []
        while lst:
            # basic search
            current_path = lst.pop() # LIFO
            current_node = current_path[-1]

            if current_node in visited:
                continue
            visited.append(current_node)

            if current_node == goal:
                return current_path, visited
            else:
                # Sort all elements alphabetically to get a unified answer !
                # Sorting in ascending order to follow "LIFO" principle !
                adjacent_nodes = self.__graph[current_node]
                adjacent_nodes.sort()
                if self.__weighted:
                    for node, _ in adjacent_nodes:
                        new_path = current_path.copy()
                        new_path.append(node)
                        lst.append(new_path)
                else:
                    for node in adjacent_nodes:
                        new_path = current_path.copy()
                        new_path.append(node)
                        lst.append(new_path)

        raise Exception("Search Failed !")

    #_____________________________________________________________________________________________
    def BFS(self, goal, start=None):
        if start != None:
            lst = [[start]]
        else:
            lst = [["root"]]

        visited = []
        while lst:
            # basic search
            current_path = lst.pop(0) # FIFO
            current_node = current_path[-1]

            if current_node in visited:
                continue
            visited.append(current_node)

            if current_node == goal:
                return current_path, visited
            else:
                # Sort all elements alphabetically to get a unified answer !
                # Sorting in ascending order to follow "FIFO" principle !
                adjacent_nodes = self.__graph[current_node]
                adjacent_nodes.sort()
                if self.__weighted:
                    for node, _ in adjacent_nodes:
                        new_path = current_path.copy()
                        new_path.append(node)
                        lst.append(new_path)
                else:
                    for node in adjacent_nodes:
                        new_path = current_path.copy()
                        new_path.append(node)
                        lst.append(new_path)

        raise Exception("Search Failed !")
```

In [2]:

```
my_graph = Graph("S", directed=True, weighted=True)
my_graph.add_vertex("A")
my_graph.add_vertex("B")
my_graph.add_vertex("C")
my_graph.add_vertex("D")
my_graph.add_vertex("G")
my_graph.add_edge("S","B", 3)
my_graph.add_edge("S","A", 2)
my_graph.add_edge("S","D", 5)
my_graph.add_edge("A","C", 4)
my_graph.add_edge("C","G", 2)
```

```
my_graph.add_edge("C","D", 1)
my_graph.add_edge("B","D", 4)
my_graph.add_edge("D","G", 5)
print(my_graph.get_graph())
```

```
Graph is created with root name: S
Vertex is added successfully !
Vertex is added successfully !
Vertex is added successfully !
Vertex is added successfully !
Vertex is added successfully !
Edge added between S and B
Edge added between S and A
Edge added between S and D
Edge added between A and C
Edge added between C and G
Edge added between C and D
Edge added between B and D
Edge added between D and G
{'S': [('B', 3), ('A', 2), ('D', 5)], 'A': [('C', 4)], 'B': [('D', 4)], 'C': [('G', 2), ('D', 1)], 'D': [('G', 5)], 'G': []}
```

In [3]:
```
print(my_graph.DFS('G', start='S'))
print(my_graph.BFS('G', start='S'))
```

```
(['S', 'D', 'G'], ['S', 'D', 'G'])
(['S', 'D', 'G'], ['S', 'A', 'B', 'D', 'C', 'G'])
```